

Lab 10

Monads and I/O in Haskell

Goals

In this lab you will learn to:

1. Understand and use Functors, Applicatives and Monads
2. Write programs that can interact with the user
3. Write programs that can work with files
4. Parse command line arguments
5. Compile your programs to native executables

Resources

Table 10.1: Lab Resources

Resource	Link
Learn You a Haskell For Great Good!	http://learnyouahaskell.com/making-our-own-types-and-typeclasses#the-functor-typeclass
	http://learnyouahaskell.com/input-and-output
	http://learnyouahaskell.com/functors-applicative-functors-and-monoids
	http://learnyouahaskell.com/a-fistful-of-monads
Functors, Applicatives and Monads in pictures	https://adit.io/posts/2013-04-17-functors,_applicatives,_and_monads_in_pictures.html
The <code>Data.Functor</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Data-Functor.html
The <code>Control.Applicative</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Control-Applicative.html
The <code>Control.Monad</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/Control-Monad.html
The <code>System.IO</code> module	https://hackage.haskell.org/package/base-4.14.0.0/docs/System-IO.html
Railway oriented programming: Error handling in functional languages by Scott Wlaschin	https://vimeo.com/113707214

10.1 Functors

10.1.1 Definition

Concept 10.1.1: Functors

Functors are type constructors that support a mapping function `fmap`, with the signature `fmap :: (Functor f) => (a -> b) -> f a -> f b`.

Functors should also respect the Functor laws:

1. `fmap id = id`
2. `fmap (f . g) = fmap f . fmap g`



Note 10.1.1

The more common form of `fmap` is its infix form: `<$>`. So instead of `fmap f d` you'll see `f <$> d`.

10.1.2 The Maybe functor

We've already discussed the `Maybe` type in the context of error handling, and saw that it has a `map` function that can be used to transform the element in the `Just` variant and ignore the `Nothing` variant.

Listing 10.1.1: Maybe Functor instance

Haskell code

```
instance Functor Maybe where
    fmap _ Nothing = Nothing
    fmap f (Just a) = Just (f a)
```

Haskell REPL

```
> fmap (+1) (Just 1)
Just 2
> (+1) <$> (Just 1)
Just 2
> fmap (++", World") (Just "Hello")
Just "Hello, World"
> fmap (-1) Nothing
Nothing
> (-1) <$> Nothing
Nothing
```

10.1.3 The List functor

We've also discussed the common functions on lists, including `map`, `filter` and `fold`. In this case you can think of `fmap` as a different name for `map`.

Listing 10.1.2: List Functor instance

Haskell code

```
instance Functor [] where
    fmap f l = map f l
```

```
> fmap (+1) [1, 2, 3]
[2, 3, 4]
> fmap ("Hello, ++") ["Alice", "Bob", "there"]
["Hello, Alice", "Hello, Bob", "Hello, there"]
> fmap (-1) []
[]
```

10.2 Applicatives

10.2.1 Definition

Concept 10.2.1: Applicatives

Applicatives generalize the idea of Functors, by allowing us to do the mapping operation with functions with more than one argument, using the `<*> :: (Applicative f) => f (a -> b) -> f a -> f b` operator. Applicatives also define a `pure :: (Applicative f) => a -> f a` function, which allows the injection of a value in the applicative.

Essentially, the `<*>` operator allows repeated partial function application inside the context of the `Applicative`. There are two options to “introduce” a function “into the applicative”:

- use `pure` to directly inject a function into the applicative
- use `fmap` to partially apply the function to a value in the applicative

10.2.2 The Maybe applicative

Listing 10.2.1: Maybe Applicative instance

Haskell code

```
instance Applicative Maybe where
  pure x = Just x
  Nothing <*> _ = Nothing
  (Just f) <*> mx = fmap f mx
```

```

> Just (+1) <*> Just 2
Just 3
> (fmap (+) (Just 1)) <*> Just 1
Just 2
> (+) `fmap` Just 1 <*> Just 1
Just 2
> (+) <$> Just 1 <*> Just 1
Just 2
> pure (+) <*> Just 1 <*> Just 1
Just 2
> (++) <$> Just "Hey, " <*> Just "you!"
Just "Hey, you!"
> (++) <$> Just "Hey" <*> Nothing
Nothing
> (\x y z -> x*100 + y*10 + z) <$> Just 1 <*> Just 2 <*> Just 3
Just 123
> pure (\x y z -> x*100 + y*10 + z) <*> Just 1 <*> Just 2 <*> Just 3
Just 123
> pure (\x y z -> x*100 + y*10 + z) <*> Just 1 <*> Nothing <*> Just 3
Nothing

```

With applicative, we can elegantly handle the case of multiple `Maybe` values.

10.2.3 The List applicative

Listing 10.2.2: List Applicative instance

Haskell code

```

instance Applicative [] where
    pure x = [x]
    fs <*> xs = [f x | f <- fs, x <- xs]

```

Haskell REPL

```

> [(+1), (*3)] <*> [1, 2, 3]
[2, 3, 4, 3, 6, 9]
> (+) `fmap` [1, 2] <*> [2, 4]
[3, 5, 4, 6]
> (+) `fmap` [] <*> [2, 4]
[]
> (\x y -> x ++ "-" ++ y) `fmap` ["Top", "Bottom"] <*> ["Left", "Right"]
["Top-Left", "Top-Right", "Bottom-Left", "Bottom-Right"]
> pure (\x y z -> x*100+y*10+z) <*> [1, 2] <*> [3, 4] <*> [5, 6]
[135,136,145,146,235,236,245,246]

```

The list applicative allows us to obtain the cartesian product of an arbitrary number of lists, with minimal effort.

10.2.4 The liftA and sequenceA functions

The `liftA2` and `liftA3` functions are just a shorter way to write some applicatives, by specifying the function and arguments directly.

```

> import Control.Applicative
> liftA2 (+) (Just 1) (Just 2)
Just 3
> liftA2 (+) Nothing (Just 1)
Nothing
> liftA2 (\x y -> x ++ "-" ++ y) ["Top", "Bottom"] ["Left", "Right"]
["Top-Left", "Top-Right", "Bottom-Left", "Bottom-Right"]
> liftA3 (\x y z -> x*100+y*10+z) (Just 1) (Just 2) (Just 3)
Just 123
> liftA3 (\x y z -> x*100+y*10+z) [1, 2] [3, 4] [5, 6]
[135,136,145,146,235,236,245,246]

```

Question 10.2.1

*

Does Elm define functions that would be the equivalent of `liftA2` and `liftA3`? What about `<*>`?

The `sequenceA` function generalizes the applicative to an arbitrary number of arguments (of the same type). Looking at its signature `sequenceA :: (Applicative f) => [f a] -> f [a]`, you can think of `sequenceA` as inverting the structure of a list and the applicative in it, if all elements are considered “present”.

```

> import Control.Applicative
> sequenceA [Just 2, Just 3, Just 4]
Just [2, 3, 4]
> sequenceA [Just 2, Just 3, Nothing, Just 5]
Nothing

```

Question 10.2.2

*

Can `sequenceA` be used in the context of error handling? If yes, how?

10.3 Monads

The main goal of Monads is to deal with *actions* or *side effects*. We’ve already seen how Elm deals with this problem: we, as the users of the programming language, only write *pure* functions and the Elm Runtime handles all the side effects in background. This will be mainly true in Haskell, but since Haskell is a much more advanced language, it also allows us to take control of this “effect management” or “state management”.

10.3.1 Definition

Concept 10.3.1: Monads

Monads introduce the bind operator `>=> :: (Monad m) => m a -> (a -> m b) -> m b` that allows the sequencing of actions. It also defines the `return` function which is the equivalent of the `pure` function from Applicative class.

On the surface level, the signature of bind (`>=>`) suggests that it allows the unpacking the

nested structures which are the results of the map operation, a description that is enough for explaining how the `Maybe` and `List` monads behave.

10.3.2 The Maybe monad

Listing 10.3.1: Maybe Monad instance

Haskell code

```
instance Monad Maybe where
    return = pure
    Nothing >>= _ = Nothing
    (Just x) >>= f = f x
```

Haskell REPL

```
> half x = if x `mod` 2 /= 0 then Nothing else Just (x `div` 2)
> half 3
Nothing
> half 2
Just 1
> (half 4) >>= half
Just 1
> (half 6) >>= half
Nothing
> (half 12) >>= half
Just 3
> (half 12) >>= half >>= half
Nothing
```

With the bind operator (`>>=`) we can easily and elegantly chain operations that can fail, writing code that looks like it only handles the “happy path”. If an error occurs (`Nothing` is returned), it is propagated forward and can be handled at the end of the chain.

10.3.3 The List monad

Listing 10.3.2: List Monad instance

Haskell code

```
instance Monad [] where
    return = pure
    xs >>= f = [fx | x <- xs, fx <- (f x)]
```

Haskell REPL

```
> [1, 2, 3] >>= (\x -> replicate x x)
[1, 2, 2, 3, 3, 3]
> [[1], [2], [3]] >>= id
[1,2,3]
```

For the list monad, the bind operator applies the mapping function that returns a list to each element and then flattens the list of lists.

10.3.4 Monads as actions

The more general way to think about monads, is that they are used to describe a sequence of actions that are executed later, in the context of some state. Using this interpretation (and given that the type variable `m` is `Monad m`), we can think of each `m a` as an action that

produces some side effect (whose interpretation depends on the type of the Monad) and a result of type `a`.

Thus, a second interpretation for the bind operator is that it allows us to sequence an action `(m a)`, and a function `(a -> m b)` that, given the result of the previous action, returns a new action `(m b)`.

Note that in order to obtain the value of `a` from `(m a)`, we must “execute” the action using a function defined for the given Monad, as in most cases the result can’t just be extracted via pattern matching.

10.3.5 The IO monad

So far we’ve only written pure functions that we tested in GHCi, but if we want to write useful applications, we need a way to communicate with the outside world (the environment the operating system offers). In Haskell this is achieved with the `IO` monad.

As with all apps, we need an entry point for our application, which in Haskell is the `main` function, which is of type `IO ()`. The `IO` means that it will interact with the outside world, and `()` means that it will return the unit value.

To print a string to the console we can use the `putStrLn` function:

```
Haskell REPL
>:t putStrLn
putStrLn :: String -> IO ()
```

which takes a `String`, prints it on the screen and also returns `IO ()`, which means that it interacts with the outside world and returns the unit value.

```
Listing 10.3.3: A Haskell app Haskell code
main :: IO ()
main = putStrLn "Hello World from Haskell"
```

Let’s try to greet the user of the application by reading their name with the `getLine` function:

```
Haskell REPL
>:t getLine
getLine :: IO String
```

This time the signature is bit more interesting, because this function only returns a value of type `String`, read from the environment. The fact that the `String` is wrapped in `IO` means that we can’t just use this function anywhere we want, because can’t obtain the value that is inside `IO`, by using pattern matching or with some function provided by the library authors.

Luckily we have the `>>=` operator whose signature `>>= :: (Monad m) => m a -> (a -> m b) -> m b` fits exactly the `getLine` and `putStrLn` functions:

```
Listing 10.3.4: Interactive Haskell app Haskell code
main :: IO ()
main = getLine >>= (\x -> putStrLn ("Hello, " ++ x))
```

The only problem with this app is that the user doesn't get any indication that they should give their name and not their age, credit card number, the title of their favorite movie or something else. So let's also ask them to enter their name:

Listing 10.3.5: Interactive Haskell app

Haskell code

```
main :: IO ()
main = putStrLn "Please enter your name:" >>=
  \_ -> getLine >>= (\x -> putStrLn ("Hello, " ++ x))
```

As you can see, because the `>>=` operator expects a function that takes one argument, but `putStrLn` returns the unit value, we needed to use a lambda to consume it. The solution is to use the `>> :: (Monad m) => m a -> m b -> m b` function, which takes two monads and returns the second, while still “running” the first one.

Listing 10.3.6: Interactive Haskell app

Haskell code

```
main :: IO ()
main = putStrLn "Please enter your name:" >>
  getLine >>= (\x -> putStrLn ("Hello, " ++ x))
```

10.4 do notation

For these small examples, the `>>=` and `>>` functions are enough, but imagine asking the user for their name, age, height and weight using this style. To make writing code with monads easy, Haskell includes *do notation*, which allows us to write imperative looking code using monads:

Listing 10.4.1: Interactive Haskell app

Haskell code

```
main :: IO ()
main = do
  putStrLn "Please enter your name:"
  name <- getLine
  putStrLn ("Hello, " ++ name)
```

The above code written using `do` notation will be *desugared* (transformed) by the compiler into the code in Listing 10.3.6:

- “statements”, like `putStrLn "Please enter your name:"`, are desugared to the `>>` function
- “bindings”, like `name <- getLine`, are desugared to the `>>=` function

You can also use if expressions as you normally would, but if you have multiple “actions” you must use the `do` keyword to start a new block with do notation:

Listing 10.4.2: Interactive Haskell app

Haskell code

```

main :: IO ()
main = do
    putStrLn "Choose a mode: toUpper or toLower"
    mode <- getLine
    if mode == "toUpper" then do
        putStrLn "Please enter the text to be converted to uppercase"
        text <- getLine
        putStrLn (map Char.toUpper text)
    else if mode == "toLower" then do
        putStrLn "Please enter the text to be converted to lowercase"
        text <- getLine
        putStrLn (map Char.toLower text)
    else
        putStrLn "Invalid mode"

```

To “run” code that is not “in” the monad (in the case of `IO` it doesn’t return `IO a`) inside `do` notation, you must use the `let` keyword to declare a new binding (note the `in` keyword is missing):

Listing 10.4.3: Interactive Haskell app

Haskell code

```

main :: IO ()
main = do
    putStrLn "Type a word to check if it's a palindrome"
    word <- getLine
    let rev = reverse word
    if word == rev then
        putStrLn "It's a palindrome"
    else
        putStrLn "Not a palindrome"

```

Finally, note that a value returned from the computation will always be the value of last expression which must be a “statement” (i.e. not a binding).

Listing 10.4.4: Using `do` notation with the `Maybe` monad

Haskell code

```

half3x n = do
    n' <- half n
    n'' <- half n'
    half n''

```

To return a value that is “not in the monad” (its type is not `m a`, for the code below `Maybe a`), we use the `return` function to “inject” the value in the context:

Listing 10.4.5: Using `do` notation with the `Maybe` monad

Haskell code

```

maybeAdd3 :: Num a => Maybe a -> Maybe a -> Maybe a -> Maybe a
maybeAdd3 ma mb mc = do
    a <- ma
    b <- mb
    c <- mc
    return (a + b + c)

```

10.5 Running and compiling Haskell programs

10.5.1 Running Haskell programs

To run Haskell programs which have a `main` function you can use the `runhaskell` command:

```
> runhaskell Main.hs
```

Shell session

10.5.2 Compiling Haskell programs

To compile a Haskell program into an executable file, you can use the `ghc` command:

```
> ghc Main.hs
```

Shell session

10.6 Working with files and command line arguments

10.6.1 Parsing command line arguments

To obtain the command line arguments passed to the program, we use the `getArgs` function, which must be imported from the `System.Environment` module. It returns a list of strings, that we can then parse into our desired structure, using functions like `read` and its safe version `readMaybe` (imported from `Text.Read`):

Listing 10.6.1 of ValidateArgs.hs (main)

Haskell code

```
6 main :: IO ()
7 main = do
8   nrStr:args <- getArgs
9   let nr = readMaybe nrStr :: Maybe Int
10  case nr of
11    Nothing -> putStrLn "First argument must be a positive number!"
12    Just n
13      | n < 0 -> putStrLn "First argument must be a positive number!"
14      | otherwise ->
15        case compare (length args) n of
16          LT -> putStrLn "Too few arguments!"
17          EQ -> putStrLn "Just enough arguments, good job!"
18          GT -> putStrLn "Too many arguments!"
```

powershell session

```
PS> runhaskell.exe .\CliArgs\ValidateArgs.hs 1 Hello
Just enough arguments, good job!
PS> runhaskell.exe .\CliArgs\ValidateArgs.hs 2 Hello
Too few arguments!
PS> runhaskell.exe .\CliArgs\ValidateArgs.hs 2 A B C
Too many arguments!
```

10.6.2 Working with files

To work with files, we will use the `readFile` and `writeFile` functions. Looking at their signature, one might think that they will simply read the whole file in memory, or need the entire string to be ready in memory before writing the file, but this is not the case, as Haskell treats IO lazily too!

What this means, is that when we write `contents <- readFile path` nothing is read! Only when `contents` is evaluated will Haskell start to read the file, and it will only read as much of it as necessary.

`writeFile` is also lazy in its own way: it will stream the data to the file as data becomes ready to be consumed, but it will stream as much as the input stream provides, so care should be taken to avoid providing infinite streams as input unless you have too much free space on your hard drive.

The final really useful function is `getContents` which reads from the Standard Input as a stream. This philosophy is used by most Linux commands, allowing the user to chain various commands together using pipelines.

The simplest example is a program that creates a copy of a file where all letters are converted to lowercase.

Listing 10.6.2 of LowerCase.hs (main)

Haskell code

```
7 main :: IO ()
8 main = do
9     [src, dst] <- getArgs
10    putStrLn $ "Copying from " ++ src ++ " to " ++ dst
11    contents <- readFile src
12    writeFile dst $ map toLower contents
13    putStrLn "Done"
```

As a second example, let's implement the Caesar¹cipher in Haskell:

First, we should implement the core logic of the programs, the `shift` function. Shifting each character by a given number is performed with the help of the `Char.ord` (get the ascii code of a character) and `Char.chr` (convert an ascii code to a character) functions. Note the function only changes characters that are letters (the check is performed using the `Char.isAlpha` function).

Listing 10.6.3 of Caesar.hs (shift)

Haskell code

```
6 shift :: Int -> Char -> Char
7 shift d c
8     | Char.isAlpha c = tr c
9     | otherwise = c where
10         tr c = (Char.chr . sh . Char.ord . Char.toLower) c
11         sh c = (c - shA + d) `mod` 26 + shA
12         shA = Char.ord 'a'
```

Then we can use this function to encrypt and decrypt files by transforming each character (by applying `shift` on each character using `map`):

¹<https://crypto.interactive-maths.com/caesar-shift-cipher.html>

Listing 10.6.4 of CaesarEncrypt.hs

Haskell code

```

1 module CaesarEncrypt where
2
3 import Caesar ( shift )
4
5 import System.Environment (getArgs)
6
7 encrypt :: Int -> String -> String
8 encrypt p c = map (shift p) c
9
10 main :: IO ()
11 main = do
12     [shAmountStr] <- getArgs
13     let shAmount = read shAmountStr :: Int
14     input <- getContents
15     putStr $ encrypt shAmount input

```

Listing 10.6.5 of CaesarDecrypt.hs

Haskell code

```

1 module CaesarDecrypt where
2
3 import Caesar ( shift )
4
5 import System.Environment (getArgs)
6
7 decrypt :: Int -> String -> String
8 decrypt p c = map (shift (26 - p)) c
9
10 main :: IO ()
11 main = do
12     [shAmountStr] <- getArgs
13     let shAmount = read shAmountStr :: Int
14     input <- getContents
15     putStr $ decrypt shAmount input

```

To confirm that logic is implemented correctly we can check the functionality in the REPL:

Haskell REPL

```

> :load CaesarEncrypt.hs CaesarDecrypt.hs
> :module CaesarDecrypt CaesarEncrypt
> key = 3
> plaintext = "To be or not to be, that is the question"
> encrypt key plaintext
"wr eh ru qrw wr eh, wkdw lv wkh txhvwlrq"
> decrypt key (encrypt key plaintext)
"to be or not to be, that is the question"
> decrypt 5 (encrypt key plaintext)
"rm zc mp lmr rm zc, rfyr gq rfc oscqrgml"

```

We can also perform the same check using a Powershell pipeline (note that you should write the whole command on one line, without `>>`, which printed by the shell because the command spans multiple lines):

powershell session

```
PS> Get-Content .\caesar.in
>> | runhaskell.exe .\CaesarEncrypt.hs 3
>> | runhaskell.exe .\CaesarDecrypt.hs 3
```

And the command prompt:

cmd session

```
>type .\caesar.in | runhaskell .\CaesarEncrypt.hs 3 | runhaskell .\CaesarDecrypt.hs 3
```

10.7 Review questions

Question 10.7.1

*

Does Elm have a function that is equivalent of `>>=`? If yes, what is its name?

Question 10.7.2

**

What statements can you make about the [theoretical] memory consumption of the following programs:

- `LowerCase.hs`
- `Words.hs`
- `RevWords.hs`
- A program that reverses the contents of a file

Which ones make best use of Haskell's laziness?

Question 10.7.3

**

Explain the `readFiles` function in the `CliArgs/Cat.hs` file.

What is the type of the `map readFile files` expression? Why do we have to use the `sequenceA` function?

Listing 10.7.1 of `Cat.hs` (`readFiles`)

Haskell code

```
23 | readFiles :: [String] -> IO [String]
24 | readFiles files = sequenceA (map readFile files)
```

10.8 Practice problems

Exercise 10.8.1

*

Implement the Functor instance for the `data Tree a = Nil | Node (Tree a) a (Tree a)` type.

Haskell REPL

```
> t = Node (Node Nil 1 Nil) 2 (Node Nil 3 Nil)
> fmap (+1) t
Node (Node Nil 2 Nil) 3 (Node Nil 4 Nil)
> fmap (*3) t
Node (Node Nil 3 Nil) 6 (Node Nil 9 Nil)
```

Exercise 10.8.2

**

Implement a function `passwords` that enumerates all `n` character passwords containing digits (0 - 9) lowercase letters (a - z) and uppercase letters (A - Z) using the list applicative.

Haskell REPL

```
> take 5 (passwords 8)
["00000000", "00000001", "00000002", "00000003", "00000004"]
> take 5 (drop 10000 (passwords 8))
["000002Bi", "000002Bj", "000002Bk", "000002Bl", "000002Bm"]
```

Hints:

You should consider using (some of) the following functions: `replicate`, `sequenceA`. Try evaluating in the REPL: `sequenceA [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`.

Exercise 10.8.3

**

Write an interactive Haskell program that asks a user for their weight (in kilograms) and height (in meters) and calculates their BMI using the following formula:

$$BMI = \frac{weight}{height^2}$$

You can assume that all the inputs will be valid.

Shell session

```
> runhaskell.exe .\Bmi.hs
Please enter your weight:
85
Please enter your height:
1.90
Your BMI is: 23.545706371191137
```

Hints:

Use the `read` function to convert a `String` to a `Double`, like `x = read "2" :: Double`. You should use `do` notation and `let` bindings.

Exercise 10.8.4

**

Write a new version of the program developed in Exercise 10.8.3 that uses command line arguments instead of reading from the standard input.

You can assume that all the inputs will be valid.

Shell session

```
> runhaskell.exe .\BmiCli.hs "85" "1.90"  
23.545706371191137
```

Exercise 10.8.5

Write a program that given a text encrypted with the Caesar cipher and a known word from the plaintext, will try to decrypt the message using brute force (by trying all possible decryptions). The known word will be provided as a command line argument and the ciphertext (encrypted message) will be read from the standard input.

powershell session

```
PS> $CipherText = "wr eh ru qrw wr eh, wkdw lv wkh txhvwlrq"  
PS> $CipherText | runhaskell.exe .\CrackCaesar.hs "question"  
Found plaintext with key: 3  
to be or not to be, that is the question  
PS> $CipherText | runhaskell.exe .\CrackCaesar.hs "that"  
Found plaintext with key: 3  
to be or not to be, that is the question  
PS> $CipherText | runhaskell.exe .\CrackCaesar.hs "attack"  
Failed to crack cipher
```

Hints:

Some questions to think about before writing the code:

- What is the keyspace of the Caesar cipher (how many unique keys are there)?
- How can we generate all possible decryptions of a given ciphertext?

You should consider writing a function that tests if a string is a substring of another string (i.e. the `strstr` function in C).

You can reuse the functions defined in the `Caesar.hs` and `CaesarDecrypt.hs` files.

Exercise 10.8.6

2 × ***

Write a pair of programs that can encrypt and decrypt files using the Vigenere cipher. The key should be read from a file provided as a command line argument and the message (plaintext for encryption and ciphertext for decryption) will be read from the standard input.

Subpoint 10.8.6.1

Implement the `VigenereEncrypt` program, that performs encryption using the Vigenere cipher.

Subpoint 10.8.6.2

Implement the `VigenereDecrypt` program, that performs decryption using the Vigenere cipher.

You can read about the Vigenere cipher here: <https://crypto.interactive-maths.>

com/vigenegravere-cipher.html

powershell session

```
> Get-Content key.txt
battista
> "a simple example" | runhaskell.exe .\VigenereEncrypt.hs .\key.txt
b lbuhee eqtuhee
> "b lbuhee eqtuhee" | runhaskell.exe .\VigenereDecrypt.hs .\key.txt
a simple example
```

Hints:

Before writing the code, you should think about the following questions:

- Are there any similarities between Vigenere and Caesar ciphers?
 - More precisely, what [mathematical] operation does each use to perform encryption and decryption?
- Can you implement encryption and decryption in a way that they use one common function (like the Caesar implementation)?

You should consider using (some of) the following functions: `zipWith`, `repeat`.

Exercise 10.8.7

Given the following data definition `data Result err ok = Err err | Ok ok`, implement the `Functor`, `Applicative` and `Monad` instances for `Result`.

Hints:

Think about the behavior of the `Result` type in Elm, and Haskell equivalents of the `Result.map` and `Result.andThen` functions.

Exercise 10.8.8

Using `Result` from Exercise 10.8.7, improve the program developed in Exercise 10.8.4 to also handle invalid inputs, by printing an error message when one of the inputs is invalid. You should validate that:

- The weight is a number between 1 and 300.
- The height is a number between 0.2 and 2.5.

powershell session

```
> runhaskell.exe .\BmiCliValidated.hs "85" "1.90"
23.545706371191137
> runhaskell.exe .\BmiCliValidated.hs "-1" "1.90"
Error: Invalid weight
> runhaskell.exe .\BmiCliValidated.hs "75" "0.01"
Error: Invalid height
```

Hints:

The `readMaybe` function from the `Text.Read` module can be used to parse a value, returning `Nothing` when the value can't be parsed instead of throwing an exception. For example, to parse a float, you can use `readMaybe "1.2" :: (Maybe Float)`.